慢在哪，以及先动哪

6,529 条记录、1,528 次调用 · 2026-11-01 · 芝士

一、实测是什么样

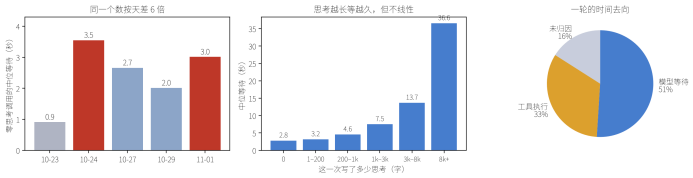


图 1　左：零思考调用的中位等待，按天拆开差 6 倍。中：写多少思考等多久。右：一轮的时间去向。

零思考的调用（几乎不生成内容）中位 **2.80 秒**，最近两天 **3.01 秒**。扣掉首块自己吐的字（约 0.5 秒），真正「不算内容」的部分是 **1.5–2.2 秒**。

它不固定：同样的工具、同样的参数规模，10-23 是 0.47–0.60 秒，10-24 是 3.17 秒，**差 6 倍**。

思考越长等越久，但不线性：0 字 2.8 秒、1–3 千字 7.5 秒、8 千字以上 36.6 秒。边际成本在 1000 字以上才明显起来。

二、三个之前不知道的

长尾吃掉四分之一：超过 30 秒的调用只占 **3.3%**，但吃掉全部模型等待的 **25%**。其中 7 次是零思考、参数几百字，却等了 66–105 秒——最长那次参数只有 338 字，是一个发消息的调用。

工具执行占三分之一：中位 0.48 秒，但占一轮墙钟的 **33%**。最慢的 1% 吃掉其中 37%；WebFetch 中位 12 秒，6 次命令行卡在 120 秒超时。

跟上下文长度没关系：用 turn 内累计字符做代理，零思考调用的等待从 0.91 到 0.99 秒几乎不变，全量回归 r^2=0.01。**所以「预填充越来越长的上下文导致慢」在我们数据里没有支持。**

三、别的 harness 记什么

Claude Code 和 Codex CLI 都把这些数交给 OpenTelemetry，**默认都关着**。Codex 也按请求记 `ttft_ms`，但源码里**没有网关侧的响应头时刻**，这一段只有我们记。

四、要做的事，按顺序

1. 首字延迟已经进记录（已做）

Claude Code 每次请求，流里那条 `message_start` 就带 `ttft_ms`，平台此前没有一处读它。人为注入 1.0 秒和 0.4 秒延迟的两轮调用，读出的正是 **1009** 和 **409** 毫秒。现在它挂在每次调用写出的那条记录上，一次调用一个数。

网关上的「上游第一个字节」量的是**网关到上游**那一段。和 `ttft_ms` 相减，就能把「网络与排队」和「预填充」分开。

2. 把已有的记录用起来

另外三样已经记着：**哪个上游**（日志带部署 id）、**token 与缓存**（计量代理的 call id 两边能对上）、**16% 的空档**（轮次表三个时间点）。打开 `CLAUDE_CODE_EMIT_STARTUP_TIMING`，还会多出 `time_to_request_ms` 的阶段拆分。缺的是 join 脚本。

3. 沙箱镜像补工具链

`backend/sandbox/Dockerfile.private` 里只有 **python3、bash、ca-certificates、ripgrep**，没有编译器、没有 poppler。一轮 21 次命令行调用里 10 次在补依赖，最后卡在共享库加载失败，**整轮零产出**。不涉及取舍，装上就行。

4. 读文件去重

同一份 `.typ` 在 4 轮里被读了 **12 次**、同一张 PNG 被读 6 次，每次都全文重进上下文。仓库里没有「本会话读过、没变就跳过」的机制。

5. 平台提醒不占轮次

「状态 10 分钟没动过」这类提醒，阈值 600 秒，每句都作为一条消息开一整轮。本会话出现十几次。

6. 工具执行没细分（测不到）

「命令跑了多久」和「平台转发开销」混在一起，分不清谁慢。

五、所以先做什么

先让第 1、2 条跑起来，把每次调用等的每一段落到具体地方。这两条不改行为，能和其他工作并行。第 3–5 条也可以一起做，但都碰不到那 25% 的长尾和 33% 的工具时间。长尾还没查清；等两段接上能分开「排队」和「预填充」，再回头看那 7 次 66–105 秒。

口径：本任务 2026-10-23 至 11-01 的 6,529 条记录、1,528 次调用。时间戳是块写完的时刻（相邻两块的差只与后一块大小相关，r^2=0.80–0.97）。`ttft_ms` 实测于 Claude Code 2.1.299。